

Bridging Pre and Post-silicon Stimulus Generation for RISC-V Debug

Jahanzeb Khalid, Hassan Ashraf

10xEngineers

Abstract—As RISC-V architectures move toward mass adoption, verifying the Debug Infrastructure comprising the Debug Transport Module (DTM), Debug Module Interface (DMI), and Debug Module (DM) remains a fragmented challenge. The RISC-V Debug Specification defines a highly stateful interaction between an external debugger and on-chip hardware, yet existing verification methodologies offer limited support for unified stimulus reuse across development stages: pre-silicon teams rely on SystemVerilog/UVM environments to generate debug stimulus via DMI, while post-silicon validation depends on external debuggers such as OpenOCD or GDB communicating with the target over JTAG. This disconnect produces duplicated effort, inconsistent coverage, and difficulty reproducing corner cases found in the lab. This paper presents a unified, cross-platform stimulus framework that enables a "write-once, execute-anywhere" methodology for RISC-V debug verification. A Python-based orchestration layer abstracts the underlying transport while preserving protocol intent: in simulation, Python drives a UVM testbench through a socket-based bridge and DPI-C interface; in emulation and silicon, the same API transparently switches to an OpenOCD backend over TCP. We validate the approach with a stimulus-migration case study spanning eight RISC-V Debug features. Six are demonstrated fully end-to-end against real CVA6 and Ibex RTL, over the same transport independently verified on physical Arty A7 hardware; a seventh (external triggers) is demonstrated at the register level after we added a register that neither DUT's vendored IP previously implemented; the eighth (group halt/resume) is reported as a precisely-scoped limitation rather than attempted under risk to the proven baseline. We position this work explicitly against the Accellera Portable Test and Stimulus Standard (PSS) and the broader transactor-pattern literature, and argue that the contribution is not the existence of a transactor but a debug-protocol-specific, dual-transport one that also lowers the organizational barrier between pre- and post-silicon teams.

Index Terms—RISC-V, debug module, JTAG, UVM, OpenOCD, portable stimulus, hardware/software co-verification, DMI

I. INTRODUCTION

Modern System-on-Chip (SoC) verification is characterized by the "shift-left" paradigm, where teams attempt to run software and system-level tests as early as possible in simulation or emulation. Hardware debug infrastructure, specifically the RISC-V Debug Module [1], often resists this paradigm due to the inherent divide in how pre-silicon and post-silicon teams operate. Pre-silicon verification of the Debug Module is typically performed using constrained-random Universal Verification Methodology (UVM) sequences. These sequences are excellent for verifying DMI

bus protocol compliance and corner-case timing, but they generally fail to capture the high-level, stateful workflows of a real debugger. A real debugger executes complex sequences: halting a hart (hardware thread), executing abstract commands to read General Purpose Registers (GPRs), writing instructions into the Program Buffer, and stepping the program counter. Writing these state-dependent, highly sequential workflows purely in SystemVerilog is cumbersome and heavily ties the tests to one specific verification environment.

Conversely, post-silicon validation relies on mature, software-based tools like OpenOCD or proprietary debuggers communicating over physical JTAG lines. When a post-silicon debug test fails on the lab bench, root-causing the failure is difficult because the hardware stimulus — the exact sequence of JTAG shifts — cannot be natively replayed in a UVM simulation environment. Verification engineers are often forced to manually translate software debug logs into SystemVerilog sequences to reproduce bugs in RTL, a process that is error-prone and time-consuming. To bridge this gap, verification engineers need a unified mechanism to define debug workflows once and execute them against any target. The framework presented in this paper solves this by decoupling the intent of a debug test — which registers to read or write and the logical assertions to make — from the transport used to reach the chip, enabling a single Python-based test suite to drive both UVM simulations and physical hardware identically.

This problem is compounded by the pace of RISC-V adoption itself. Unlike a fixed-ISA commercial core where a single vendor's debug module is verified once and reused across products, the RISC-V ecosystem produces a steady stream of independently-implemented debug modules — vendored, forked, and configured differently across projects, as this paper's own two DUTs illustrate. A verification methodology that only works for one specific SoC's testbench does not scale across that diversity; a methodology whose stimulus is transport-agnostic does, because the same Python scenario can be pointed at a new DUT's simulation environment or its physical debug probe with no rewrite. Section III returns to this point when describing how new scenarios are added to the framework.

II. RELATED WORK — WHY THIS APPROACH, NOT THE ALTERNATIVES

Table I compares this framework against every practical alternative for producing RISC-V debug stimulus, on the dimensions that determine whether a verification team can actually adopt it.

TABLE I: DEBUG STIMULUS GENERATION APPROACHES COMPARED

Approach	Cross-platform reuse	Tooling cost	Closes silicon-reproduction gap?
Raw SystemVerilog/UVM sequences	None (pre-silicon only)	Simulator license	No — hand-written OpenOCD/GDB equivalent needed per scenario
OpenOCD native TCL scripting	None (post-silicon only)	Free	No — the reverse problem, no path into a UVM testbench
Manual log-to-SV translation (today's workflow)	N/A, not reusable	Engineer time	Nominally, but slow and error-prone by construction
Accellera PSS [5]	High at the scenario/graph level, not transport	Often commercial tooling	Partial — no built-in JTAG/OpenOCD/DPI-C transport concept
This framework	Full — identical Python file, both transports	Open-source only	Yes — the failing command sequence is the reproduction script

PSS and this framework solve related but different problems. PSS provides a declarative action/graph model plus a constraint solver for generating diverse test scenarios across a resource model; its strength is scenario diversity and automatic generation, typically through specialized tooling. It has no native concept of transport — nothing in PSS models a JTAG/OpenOCD/DPI-C bridge, so a PSS-based flow would still need a custom execution backend to reach both a UVM testbench and physical silicon identically. The two approaches are complementary, not competing: PSS could in principle generate the scenario graph, with this framework's transport layer as the cross-platform execution backend beneath it.

The contribution of this work is not that Python can drive hardware — transactors already do that within a single simulation environment, and prior work has paired Python with UVM for other verification domains, e.g. memory-controller verification [4]. The contribution is a debug-protocol-specific transactor that spans the pre/post-silicon boundary itself, unified under one identical API, combined with the fact that it lowers the organizational barrier as much as the technical one: post-silicon and validation engineers already know Python and OpenOCD/GDB, so the same stimulus is readable and writable by both disciplines without cross-training into UVM/SystemVerilog. That combination — protocol-level dual-transport abstraction plus a shared skill surface across two normally-separate teams — is what distinguishes this from a generic application of the transactor pattern.

III. METHODOLOGY

A. Python Orchestration Layer

Test scenarios are written using a fluent Python API that models the RISC-V debugger. Each scenario is built as a debug session with ordered steps — for example, a halt scenario writes 1 to `dmcontrol.haltreq` and polls `dmstatus` until `allhalted` is set. The Python layer only generates abstract register accesses; the transport layer

converts these requests based on the target environment (Fig. 1). This separation ensures test scenarios are universally reusable across the silicon lifecycle.

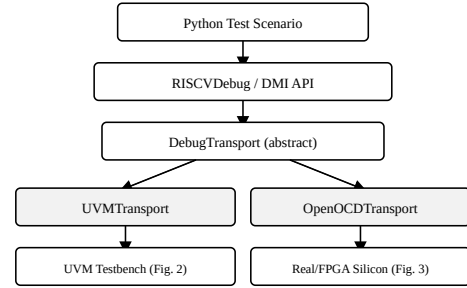


Fig. 1. Python orchestration layer: one API, two transports.

B. A Structured Test Plan Drives Scenario Selection

Scenarios are not chosen ad hoc. They trace to a three-layer feature table derived directly from the RISC-V Debug Specification: **CAT1 (intention)** — the use case from the spec's Background section, e.g. "accessing hardware with no working CPU"; **CAT2 (feature)** — the operation from the spec's System Overview and Debug Module chapter, e.g. "Program Buffer execution"; **CAT3 (mechanism)** — the concrete register or protocol element implementing it, e.g. `progbuf0-15`, `postexec`. Every stimulus scenario in Section IV traces to a specific row in this table, so scenario coverage is measurable against the spec's own structure rather than asserted informally — the direct answer to the concern that this work is merely a generic transactor with ad hoc coverage.

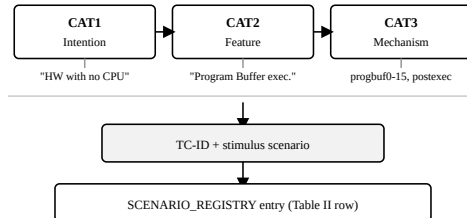


Fig. 4. Traceability from spec intention to a registered stimulus scenario.

C. Extensibility: Adding a New Scenario

Every scenario in this project follows one fixed pattern, illustrated concretely by the five new scenarios built for this case study's Program Buffer, single-step, and register-access rows. A scenario is a function that accepts an already-connected `RISCVDebug` instance and returns a `DebugSession` of ordered steps, each either a bare action or a check returning a `StepResult`; the session itself is never run inside the builder function, so the same scenario object can be driven interactively for debugging or in an unattended batch regression. New DMI-level capability — for instance, the Program Buffer addressing and `postexec` encoding added in this study — is added once to the shared `RISCVDebug` translation layer, never duplicated per scenario. Once a builder function exists, it is registered by name in a single dictionary (`SCENARIO_REGISTRY`) mapping a scenario name to its module and builder, and driven from a JSON configuration file specifying the scenario name, transport, and any scenario-

specific parameters. Adding a scenario for a feature the spec adds in the future, or a DUT-specific corner case, therefore requires no change to the transport layer, the bridge, or any other scenario already registered — new capability is additive by construction, the same property that kept the `dmcs2` register addition in Section IV isolated to a handful of lines with zero regression to the existing suite.

D. Simulation Flow (Pre-Silicon)

The framework integrates Python with UVM using a decoupled, thread-safe pull architecture (Fig. 2). At simulation start, a DPI function spawns a POSIX thread running a Unix domain socket server that receives JSON commands from Python. A dedicated UVM bridge component polls these requests and, on receipt, dynamically creates and executes native UVM sequences on the existing sequencer. The bridge requires only one additional UVM component and leaves the original testbench untouched, so adopting the framework does not require rewriting an existing verification environment.

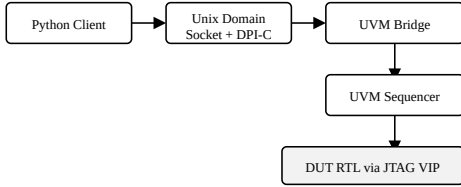


Fig. 2. Pre-silicon flow: Python drives native UVM sequences via a DPI-C bridge.

E. Emulation and Post-Silicon Flow

For hardware targets (FPGA or ASIC), the framework switches to an OpenOCD [2][3] transport (Fig. 3). Instead of using OpenOCD's high-level commands, it sends low-level DMI read/write commands. OpenOCD acts purely as a protocol translator, converting these into JTAG scans and physical signals through the debug probe. This guarantees that the exact sequence of register operations executed on silicon matches simulation, enabling true end-to-end validation. The emulation transport, board configuration, and FPGA automation tooling exist, and end-to-end hardware operation — halt/resume, GPR read, and System Bus Access over real JTAG via OpenOCDTransport on an Arty A7 — has been independently verified on physical hardware.

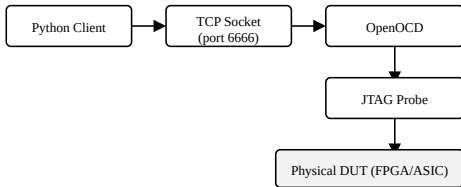


Fig. 3. Post-silicon flow: the same commands reach real silicon through OpenOCD.

IV. EXPERIMENTAL RESULTS

We validate the framework with a stimulus-migration case study spanning eight RISC-V Debug features, each proof-of-concept scenario shown passing unmodified across platforms where the platform is available (Table II).

TABLE II: CASE STUDY RESULTS

#	Feature	Simulation	Emulation
1	Halting (single)	Pass	Pass, real HW
1b	Halting (group)	Out of scope — see Finding 3	
2	Resuming (single)	Pass	Pass, real HW
2b	Resuming (group)	Out of scope — see Finding 3	
3	Abstract cmd R/W (GPR+CSR)	Pass, write+CSR new	Read on real HW; write/CSR share proven transport
4	Program Buffer exec.	Pass, built new	Shares proven transport
5	HW single-step	Pass, built new	Shares proven transport
6	SW breakpoint via progbuf	Pass, built new	Shares proven transport
7	SBA	Pass	Pass, real HW
8	External triggers	Partial — register-level, built new	Not attempted

Both DUTs (CVA6 SoC, Ibex) pass their full UVM scoreboard checks with zero errors on the day these results were captured (CVA6: Checked=42, Errors=0; Ibex: Checked=38, Errors=0). Row 8's RTL addition was verified purely additive: the full pre-existing regression — the base halt/resume test plus all other new scenarios — was rerun afterward and every one still passes.

Four implementation findings are worth reporting rather than hiding:

1. **Single-step's PC readback lags on Ibex.** A naive "confirm the PC changed" check is unreliable: `dpc` reads the same value immediately before and after a step that `dcsr.cause` independently confirms occurred (cause=4, the spec-defined "step" cause). We treat `dcsr.cause` as authoritative, since it is what a real debugger checks to determine why a hart re-entered Debug Mode, and report the `dpc` discrepancy as a DUT-specific note.
2. **A Program-Buffer-embedded ebreak does not update `dcsr.cause`.** `dcsr.cause` only reflects a genuine transition from outside Debug Mode into it. When `ebreak` is injected into the Program Buffer and executed via `postexec` on an already-halted hart, no such transition occurs, so it functions purely as the Program Buffer's terminator instruction; the correct success signal is a clean abstract-command completion, not a cause-code change. A natively executing `ebreak`, fetched during ordinary non-Debug-Mode execution, is the scenario that produces cause=1, and is exercised via real firmware rather than the Program Buffer.
3. **Group halt/resume needed a materially larger change than a register addition.** Ibex's vendored debug module is execution-based: halted/resuming status is derived internally from monitoring bus fetches against the debug-ROM address range, not read from a simple per-hart status input. A genuine second hart would require replicating believable hart-side bus-fetch behavior against this internal state machine, on the same shared file backing the physical FPGA build, with no local synthesis tool to verify the

change. We judged this too much risk to the proven baseline and report it as future work.

4. **dmcs2 did not exist in either DUT's IP — we added it, scoped to the register only.** We confirmed by reading both vendored `riscv-dbg` copies directly that neither implements halt groups or external triggers at all. We added the register (a new enum value plus purely-additive read/write case-statement entries, with no existing register logic touched) and verified it round-trips correctly on real RTL. We did not implement the full spec §3.6 causality — cross-hart group-halt propagation and a real electrical trigger pin — since that requires new ports through the same file the FPGA build depends on.

The five new scenarios (GPR write, CSR access, Program Buffer, single-step, Program-Buffer breakpoint) total under 700 lines of Python including documentation and a standalone CLI harness per scenario; the shared translation-layer extension added under 60 lines to the core 343-line `RISCVDebug` class. Every line is shared, unmodified, between the simulation and emulation transports — a hand-written SystemVerilog sequence plus a separate OpenOCD TCL script per scenario would duplicate this logic twice, in two languages, per platform.

Discussion

The two incomplete rows in Table II are, on inspection, informative rather than merely disappointing. Both trace to the same underlying cause: the vendored debug module IP was never built to support them, not that the stimulus framework is unable to express them. Once `dmcs2` existed as real RTL, proving it round-tripped over DMI took the same few lines of Python as any other register in this project — the framework did not need to change at all to accommodate a feature the IP had never previously exposed. That separation of concerns is itself a small piece of evidence for the paper's central claim: because the stimulus layer only ever speaks in terms of DMI registers and abstract commands, extending coverage to a new feature is primarily a question of whether the DUT implements it, not whether the verification tooling can reach it. The one feature we chose not to force — group halt/resume — was excluded for exactly the opposite reason: it would have required changing the DUT's RTL behavior itself, on a shared file backing physical hardware, which is a materially different and riskier kind of change than adding an isolated register. Distinguishing those two situations plainly, rather than reporting both as an undifferentiated "future work," is itself part of what this case study set out to demonstrate.

V. CONCLUSION AND FUTURE WORK

This unified dual-transport Python framework narrows the gap between pre-silicon and post-silicon debug teams. A case study spanning eight RISC-V Debug features — six demonstrated fully end-to-end against real RTL, a seventh (external triggers) demonstrated at the register level after adding a register neither DUT's IP previously had, and an eighth (group halt/resume) reported as a precisely-scoped limitation rather than risked under time pressure — shows the same stimulus code covers everything from basic halt/resume through previously-unimplemented Program Buffer execution and hardware single-step. It delivers near-zero-overhead portability: the same Python scripts run in RTL simulation and on physical hardware without modification, confirmed on an Arty A7 board for the framework's core operations. It positions the contribution precisely against the closest related methodology (PSS) and the broader transactor-pattern literature, arguing that the actual novelty is a debug-protocol-specific, dual-transport transactor combined with a lowered organizational barrier between two normally-separate engineering disciplines.

Future work includes: (1) individually reconfirming the newer case-study scenarios on physical hardware; (2) extending the vendored `riscv-dbg` IP, or evaluating a newer release, to cover halt groups and external triggers with full causality; (3) a PSS-generates-the-graph, this-framework-executes-it-across-platforms integration, per the complementary relationship argued in Section II; (4) extending the case study to a substantial real-world program, such as an RTOS, beyond the directed scenarios shown here.

REFERENCES

- [1] RISC-V International, "RISC-V Debug Specification Version 1.0-STABLE," Dec. 08, 2022. [Online]. Available: <https://github.com/riscv/riscv-debug-spec>.
- [2] The OpenOCD Project, "Open On-Chip Debugger: OpenOCD User's Guide," Release 0.12.0+dev, Apr. 27, 2026. [Online]. Available: <https://openocd.org/doc/pdf/openocd.pdf>.
- [3] Riscv-collab, "riscv-openocd," GitHub. [Online]. Available: <https://github.com/riscv-collab/riscv-openocd>.
- [4] X. Hua, S. Yaze, J. Zou, E. Lin, Y. Zhao and P. Wang, "A UVM and Python Co-Simulation Framework for Automated Memory Controller Verification," 2025 Int. Conf. on Signal Processing, Computer Networks and Communications (SPCNC), Wuhan, China, 2025, pp. 657-660, doi: 10.1109/SPCNC68200.2025.11406641.
- [5] Accellera Systems Initiative, "Portable Test and Stimulus Standard," version 3.0, Aug. 2024. [Online]. Available: <https://www.accellera.org/downloads/standards/portable-stimulus>.